Wikipedia Path Finder Project Report

Administrative information

Team name: [Enter official team name]

Team members: [Full name - GitHub username]; [Full name - GitHub username]; [Full name - GitHub username]

GitHub repository: https://github.com/GrouganDev/DSA-Project-2

Video demonstration: [Paste the public or instructor-accessible video URL]

Extended and refined proposal

Problem

Finding a short sequence of hyperlinks between two Wikipedia articles is difficult to do manually. Our project, wikiruiner, models articles as graph vertices and hyperlinks as directed edges. Given a starting title and a destination title, it returns a route with the fewest hyperlinks in the loaded dataset, or reports that no directed route exists.

Motivation

The project makes graph traversal visible through a familiar collection of articles. It also lets us compare breadth-first search (BFS) with Dijkstra on the same unweighted graph. Both algorithms should produce the same minimum link count, although equally short routes and execution times may differ.

[Add 1-2 sentences explaining your group's personal interest in this problem.]

Features implemented

The console application accepts repeated title-based searches. The Qt desktop interface provides starting and destination fields, an algorithm selector, a horizontal path diagram, actual graph statistics, algorithm timing, recent routes, and links to Wikipedia articles. Loading and search run in background tasks. The interface validates missing titles and graph membership and supports retrying after missing data files.

Extended proposal continued

Description of data

The source is the SNAP English Wikipedia hyperlink network from a 2013 snapshot [1]. The local binary graph contains 1,703,506 directed edge records and 99,195 distinct article IDs. Each record stores two integers: source and destination. The separate SQLite names database contains 4,206,745 title records, with IDs ranging from 0 to 4,206,784 and some gaps. These local counts describe different files; the title database is larger than the searchable graph.

Tools languages and libraries

C++ implements graph loading, searches, and the console interface. The GUI uses C++17 with Qt Quick/QML, Quick Controls 2, Qt Concurrent, and QElapsedTimer. SQLite supplies local title/ID queries. CMake builds the programs; Git and GitHub support collaboration. Figma supplied the interface wireframe. The delivered GUI is a desktop application [2], not a hosted website.

Algorithms and additional data structures

BFS uses a FIFO queue, a visited set, and a parent map. Dijkstra uses a min-priority queue, distance and parent maps, and unit edge weights. Both reconstruct the result by following parents backward and reversing a vector. An unordered map of unordered sets stores outgoing neighbors and removes duplicate links. Hash sets also support graph statistics and membership checks. SQLite stores titles separately from the in-memory graph.

Distribution of responsibility and roles

[Member 1 name / username]: [Describe owned files/features, implementation, testing, and documentation contributions.]

[Member 2 name / username]: [Describe specific contributions.]

[Member 3 name / username]: [Describe specific contributions. Add or remove members as needed.]

Shared work: [Describe integration, review, meetings, report writing, and video preparation.]

Analysis

Changes after the original proposal

Original plan: [Summarize the actual submitted proposal in 1-2 sentences.]

Change and rationale: [State a specific difference, why it was needed, and its effect. Possible changes to confirm against your proposal include the smaller dataset, C++ database integration, adjacency sets, or the Qt interface. Do not describe a change as proposed unless it actually was.]

Worst case time and space complexity

Let V be distinct graph vertices, E stored directed edges, R input edge records, N database rows, and L vertices in a returned path. The following traversal bounds assume expected constant-time hash access; they are worst case over the portion of the graph a query must explore. String comparison and output lengths are treated as bounded.

Graph construction: Expected amortized O(R) time to insert edge records. Persistent storage is O(V + E); the current temporary edge array makes peak memory O(R + V + E).

BFS: O(V + E) time and O(V) extra space. Each reachable vertex is enqueued once and its outgoing links are scanned. Parent reconstruction and vector reversal take O(L) time.

Dijkstra: O((V + E) log V) is a valid binary-heap upper bound. With every edge costing one, the current implementation can be bounded more tightly by O(E + V log V): a vertex's first discovered distance is shortest. Extra space is O(V) for this unit-weight case.

Strict hash-collision worst case: Unordered containers do not guarantee O(1) access. Graph construction can degrade to O(RV). BFS can degrade to O(V(V + E)); Dijkstra to O(V(V + E) + V log V). These distinguish implementation worst cases from standard expected-hash bounds.

Membership and statistics: Checking whether an ID appears as a key or destination takes O(V) expected time and O(V + E) under collisions. Counting distinct vertices and links takes O(V + E) expected time with O(V) temporary space.

Analysis continued

Database and display costs

The current case-insensitive title-to-ID query performs a table scan in the supplied database, so a lookup is O(N). Inspection with EXPLAIN QUERY PLAN confirmed SCAN names. ID-to-title lookup uses the integer primary key and is O(log N) [3]. Resolving a path's titles is O(L log N); creating L diagram nodes is O(L). A matching NOCASE index could improve title lookup but is not currently implemented.

Validation and limitations

Integration checks cover both algorithms, unreachable destinations, identical endpoints, destination-only vertices, invalid titles, articles outside the graph, missing data and retry, and the GUI button. A real-data example, Seung-min to Frans Kokshoorn, returns 18 articles and 17 links with both searches. Different tie-breaking can produce different routes of the same length.

Displayed time measures the algorithm call, excluding title lookup and drawing. The page-limit slider is disabled because the algorithms accept no limit. The summary reports loaded article counts, not explored-vertex counts. Data is historical and incomplete; titles in names.db are not necessarily present in the smaller graph. The console Database class still needs a review of its sqlite3_step result checks; GUI queries check SQLITE_ROW explicitly.

Reflection

Overall group experience

[Write 3-4 sentences describing how the group worked together, what went well, and how the completed application compares with your expectations. Use your actual experience.]

Challenges and resolutions

[Describe 1-2 real challenges. Explain what happened, how you diagnosed it, how you addressed it, and any unresolved limitation. Possible topics include missing files, compiler/Qt setup, graph direction, unreachable-path handling, or connecting QML to C++.]

Reflection continued

What we would change next time

[Write 3-4 sentences about changes to the project or workflow. Consider agreeing on interfaces earlier, documenting setup sooner, testing edge cases before GUI integration, using smaller test fixtures, reviewing pull requests, or assigning integration responsibilities. Explain why your selected changes would help.]

What each member learned

[Member 1]: [In 2-3 sentences, identify a concrete technical lesson and a collaboration lesson. Connect them to work this member actually completed.]

[Member 2]: [Describe this member's learning and supporting example.]

[Member 3]: [Describe this member's learning and supporting example. Adjust the number of entries to match your group.]

References

[1] Stanford Network Analysis Project. English Wikipedia hyperlink network. https://snap.stanford.edu/data/enwiki-2013.html

[2] The Qt Company. Qt Quick documentation. https://doc.qt.io/qt-6/qtquick-index.html

[3] SQLite. Query Planning. https://www.sqlite.org/queryplanner.html

[4] Project source code and revision history. https://github.com/GrouganDev/DSA-Project-2